06-4-egress-control

accounts-api security assessment · generated from the repository, not hand-edited

```
# 06-4-egress-control.yaml
#
# Section 6 deliverable: the egress control — default-deny at L3/L4 plus an
# FQDN- and CA-pinned egress gateway at L7. Documented limitations are in
# 06-6-egress-limitations.md.
#
# Assembled by scripts/build-submission.py from the repository.

# ============================================================================
# SOURCE: k8s/networkpolicy-egress.yaml
# ============================================================================
# Layer 1 of the egress control: default-deny at L3/L4. Threats T5, T2.
# Rationale and the honest list of what this still fails to stop: docs/05-egress.md
---
# Namespace-wide default deny. This goes in FIRST and on its own, because a
# default-deny that lands at the same time as its allow rules is a rollback
# waiting to happen and nobody can tell which rule broke the service.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
  namespace: accounts
spec:
  podSelector: {}
  policyTypes: [Egress]
  # no egress: block — every destination must be named below
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: accounts-api-egress
  namespace: accounts
  annotations:
    example-bank.io/threat: "T5 — data egress; T2 — C2 channel"
spec:
  podSelector:
    matchLabels:
      app: accounts-api
  policyTypes: [Egress]
  egress:
    # DNS. Restricted to CoreDNS pods rather than "UDP/53 anywhere", because
    # UDP/53 to the world is a serviceable exfiltration channel and a very
    # common hole in otherwise tight egress policies.
    - to:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: kube-system }
          podSelector:
            matchLabels: { k8s-app: kube-dns }
      ports:
        - { protocol: UDP, port: 53 }
        - { protocol: TCP, port: 53 }

    # The KYC call does NOT go direct. It goes to the Istio egress gateway, which
    # is where the FQDN and the certificate are pinned (istio-egress.yaml).
    # A NetworkPolicy cannot express "only this hostname" — it works on IPs, and
    # a SaaS provider's IPs change without notice.
    - to:
```

```
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: istio-system }
          podSelector:
            matchLabels: { istio: egressgateway }
      ports:
        - { protocol: TCP, port: 443 }

    # RDS and MSK, by CIDR because they are ENIs in our own private subnets and
    # those CIDRs are ours and stable.
    - to:
        - ipBlock: { cidr: 10.42.32.0/20 }   # data subnets: RDS
      ports:
        - { protocol: TCP, port: 5432 }
    - to:
        - ipBlock: { cidr: 10.42.48.0/20 }   # msk subnets
      ports:
        - { protocol: TCP, port: 9094 }      # TLS listener only; 9092 plaintext is not permitted
---
# Ingress side: only the ALB target group reaches the pods. Without this, any of
# the twelve co-tenant services can call accounts-api directly and skip the WAF,
# the ALB's mTLS, and the edge rate limits — which is how "authenticated API"
# quietly becomes "authenticated at the edge only".
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: accounts-api-ingress
  namespace: accounts
spec:
  podSelector:
    matchLabels: { app: accounts-api }
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: ingress }
          podSelector:
            matchLabels: { app: aws-load-balancer-controller-target }
      ports:
        - { protocol: TCP, port: 8080 }
    - from:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: monitoring }
          podSelector:
            matchLabels: { app: prometheus }
      ports:
        - { protocol: TCP, port: 9090 }

# ============================================================================
# SOURCE: k8s/istio-egress.yaml
# ============================================================================
# Layer 2 of the egress control: L7 identity of the destination. Threat T5.
#
# Why this layer exists at all: a NetworkPolicy allows IPs, and the KYC provider
# is a SaaS behind a CDN whose IPs change without notice. Allowing their current
# /24 means either (a) a broken integration the week they re-IP, or (b) an
# allowlist so wide it includes every other tenant of the same CDN. Neither is an
# egress control. Pinning the FQDN and the server certificate is.
---
apiVersion: networking.istio.io/v1beta1
kind: ServiceEntry
metadata:
  name: kyc-provider
  namespace: accounts
spec:
```

```
  hosts:
    - kyc.example-kyc.com
  ports:
    - number: 443
      name: https
      protocol: TLS
  resolution: DNS
  location: MESH_EXTERNAL
---
# REGISTRY_ONLY is the line that makes this a deny-by-default: anything not in the
# mesh registry is refused by the sidecar, rather than passed through.
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
metadata:
  name: mesh-egress-posture
spec:
  meshConfig:
    outboundTrafficPolicy:
      mode: REGISTRY_ONLY
---
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: kyc-egress
  namespace: istio-system
spec:
  selector:
    istio: egressgateway
  servers:
    - port:
        number: 443
        name: tls
        protocol: TLS
      hosts:
        - "accounts/kyc.example-kyc.com"
      tls:
        mode: PASSTHROUGH
---
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: kyc-through-egress-gateway
  namespace: accounts
spec:
  hosts:
    - kyc.example-kyc.com
  gateways:
    - mesh
    - istio-system/kyc-egress
  tls:
    - match:
        - port: 443
          sniHosts: [kyc.example-kyc.com]
          gateways: [mesh]
          sourceLabels: { app: accounts-api }   # only this workload, not the namespace
      route:
        - destination:
            host: istio-egressgateway.istio-system.svc.cluster.local
            port: { number: 443 }
    - match:
        - port: 443
          sniHosts: [kyc.example-kyc.com]
          gateways: [istio-system/kyc-egress]
      route:
```

```
        - destination:
            host: kyc.example-kyc.com
            port: { number: 443 }
---
# Certificate pinning at the egress gateway. Without this, an attacker with DNS
# control (or a mis-issued certificate from any of the ~150 CAs in the default
# trust store) satisfies the FQDN allowlist and receives the customer data.
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: kyc-provider-tls
  namespace: istio-system
spec:
  host: kyc.example-kyc.com
  trafficPolicy:
    portLevelSettings:
      - port: { number: 443 }
        tls:
          mode: SIMPLE
          sni: kyc.example-kyc.com
          # CA bundle restricted to the provider's issuing CA, not the system store.
          caCertificates: /etc/istio/kyc-ca/ca.pem
          subjectAltNames: ["kyc.example-kyc.com"]

# ============================================================================
# SOURCE: k8s/networkpolicy-quarantine.yaml
# ============================================================================
# Used by step 3 of runtime/runbook-accounts-api.md. Labelling a pod
# `quarantine=true` cuts it off at the network layer without deleting it, so the
# process tree, open file descriptors and memory survive for forensics.
#
# Committed rather than improvised at 03:00, because the version somebody types
# during an incident is the version that has a typo in the podSelector.
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: quarantine
  namespace: accounts
spec:
  podSelector:
    matchLabels:
      quarantine: "true"
  policyTypes: [Ingress, Egress]
  # no ingress and no egress rules: total isolation.
  #
  # Caveat the responder must know: this does NOT revoke the pod's AWS
  # credentials. IRSA tokens are already minted and the STS session outlives the
  # network cut. Rotating the secret (step 1) and, if needed, attaching a Deny-all
  # inline policy to the pod role is what stops the AWS side. Network isolation
  # stops the C2 channel, not the identity.
```